Git Cheat Sheet

Basics

`git init` Initializes a new Git repository

Usage: `git init`
Example: `git init`
Explanation:
This command creates a new Git repository in your current directory. It creates a hidden .git directory that contains all Git metadata.

`git status` Shows the status of the repository

Usage: `git status`
Example: `git status`
Explanation: This lets you see the current status of your repository - which files have been changed, which are staged, etc.

`git add` Adds file contents to the index

Usage: `git add <file> or git add .`
Example: `git add index.html`
Explanation:
With this command, you mark changes for the next commit. Use 'git add .' to mark all changes in the current directory.

`git commit` Records changes to the repository

Usage: `git commit -m "<message>"`
Example: `git commit -m "Fix bug in login form"`
Explanation:
Creates a new commit with all staged changes. The message should be a short, precise description of what was changed.

`git config` Configure Git settings

Usage: `git config [--global] <key> <value>`
Example: `git config --global user.name "Your Name"`
Explanation:
Sets configuration values for your user name, email, editor, and other preferences. Use --global to apply settings to all repositories.

`git help` Display help information

Usage: `git help <command>`
Example: `git help commit`
Explanation:
Shows detailed help information for any Git command. You can also use 'git <command> --help' for the same information.

Branches

`git branch` Lists, creates, or deletes branches

Usage: `git branch [name] [--delete]`
Example: `git branch feature-login`
Explanation:
Without parameters, this command lists all existing branches. With a name, it creates a new branch (but doesn't switch to it).

`git checkout` Switches branches or restores files

Usage: `git checkout <branch> or git checkout -b <new-branch>`
Example: `git checkout -b feature-login`
Explanation: Switches to another branch. With '-b', it creates a new branch and immediately switches to it.

`git merge` Joins two or more development histories

Usage: `git merge <branch>`
Example: `git merge feature-login`
Explanation:
Integrates changes from the specified branch into the current branch. This creates a merge commit if it's not a fast-forward situation.

`git switch` Switch to a specified branch

Usage: `git switch <branch> or git switch -c <new-branch>`
Example: `git switch main`
Explanation:
Modern alternative to 'git checkout' for switching branches. Use '-c' to create and switch to a new branch in one command.

`git branch -d` Delete a branch

Usage: `git branch -d <branch>`
Example: `git branch -d feature-login`
Explanation:
Deletes the specified branch if it has been fully merged. Use '-D' instead of '-d' to force deletion even if not merged.

Commit History

`git log` Shows the commit history

Usage: `git log [options]`
Example: `git log --oneline --graph`
Explanation:
Shows the commit history with details like author, date, and message. Many options available to customize the output format.

`git diff` Show changes between commits

Usage: `git diff [<commit>] [<commit>]`
Example: `git diff HEAD~1 HEAD`
Explanation:
Shows the differences between two commits, commit and working tree, etc. Without arguments, shows changes in working directory that aren't staged.

`git show` Show various Git objects

Usage: `git show [<commit>]`
Example: `git show HEAD`
Explanation: Shows information about a git object. For commits, shows the commit message and the differences it introduced.

`git blame` Show who changed what in a file

Usage: `git blame <file>`
Example: `git blame index.html`
Explanation:
Shows who made each change to a file, line by line, and in which commit. Useful for understanding the history of a specific file.

Remote Repositories

`git clone` Clones a repository into a new directory

Usage: `git clone <url>`
Example: `git clone https://github.com/user/repo.git`
Explanation: Creates a local copy of a remote repository, including all branches and history.

`git pull` Fetches and integrates changes from a remote repository

Usage: `git pull [remote] [branch]`
Example: `git pull origin main`
Explanation:
Combines 'git fetch' and 'git merge' to fetch changes from a remote branch and integrate them into your current branch.

`git push` Updates remote references and associated objects

Usage: `git push [remote] [branch]`
Example: `git push origin main`
Explanation: Sends your local commits to a remote repository. Others can then see and fetch your changes.

`git remote` Manage remote repositories

Usage: `git remote add <name> <url>`
Example: `git remote add origin https://github.com/user/repo.git`
Explanation: Lists, adds, or removes remote repositories. Use 'git remote -v' to see the URLs of your remotes.

`git fetch` Download objects and refs from remote

Usage: `git fetch [remote]`
Example: `git fetch origin`
Explanation: Downloads all branches and commits from a remote repository without merging them into your local branches.

Undoing Changes

`git restore` Restore working tree files

Usage: `git restore <file> or git restore --staged <file>`
Example: `git restore --staged index.html`
Explanation: Undoes changes to your working tree (with no options) or removes files from the staging area (with --staged).

`git reset` Reset current HEAD to a specific state

Usage: `git reset [--soft | --mixed | --hard] [commit]`
Example: `git reset --hard HEAD~1`
Explanation:
Resets your branch to a specific commit. --soft keeps changes in staging area, --mixed (default) unstages them, --hard discards all changes.

`git revert` Create commit that undoes changes

Usage: `git revert <commit>`
Example: `git revert HEAD`
Explanation: Creates a new commit that undoes the changes made by an earlier commit. Safer than reset for shared branches.

Advanced Commands

`git rebase` Reapplies commits on top of another base

Usage: `git rebase <base>`
Example: `git rebase main`
Explanation: Transfers your changes onto the latest version of the base branch. This creates a cleaner history than merges.

`git stash` Stashes changes temporarily

Usage: `git stash [pop]`
Example: `git stash`
Explanation:
Saves your uncommitted changes temporarily, allowing you to return to a clean working directory. Use 'pop' to reapply the stashed changes.

`git tag` Create, list, delete tags

Usage: `git tag [name] [commit]`
Example: `git tag v1.0.0`
Explanation:
Tags are references to specific points in Git history, typically used for marking version releases. Use with -a for annotated tags.

`git cherry-pick` Apply changes from specific commits

Usage: `git cherry-pick <commit>`

Example: `git cherry-pick abc123`

Explanation:

Applies the changes from specific commits to your current branch. Useful for selectively bringing changes from one branch to another.

`git bisect` Use binary search to find bugs

Usage: `git bisect <subcommand>`

Example: `git bisect start`

Explanation:

Helps find which commit introduced a bug using binary search. Mark commits as 'good' or 'bad' to narrow down the problem.